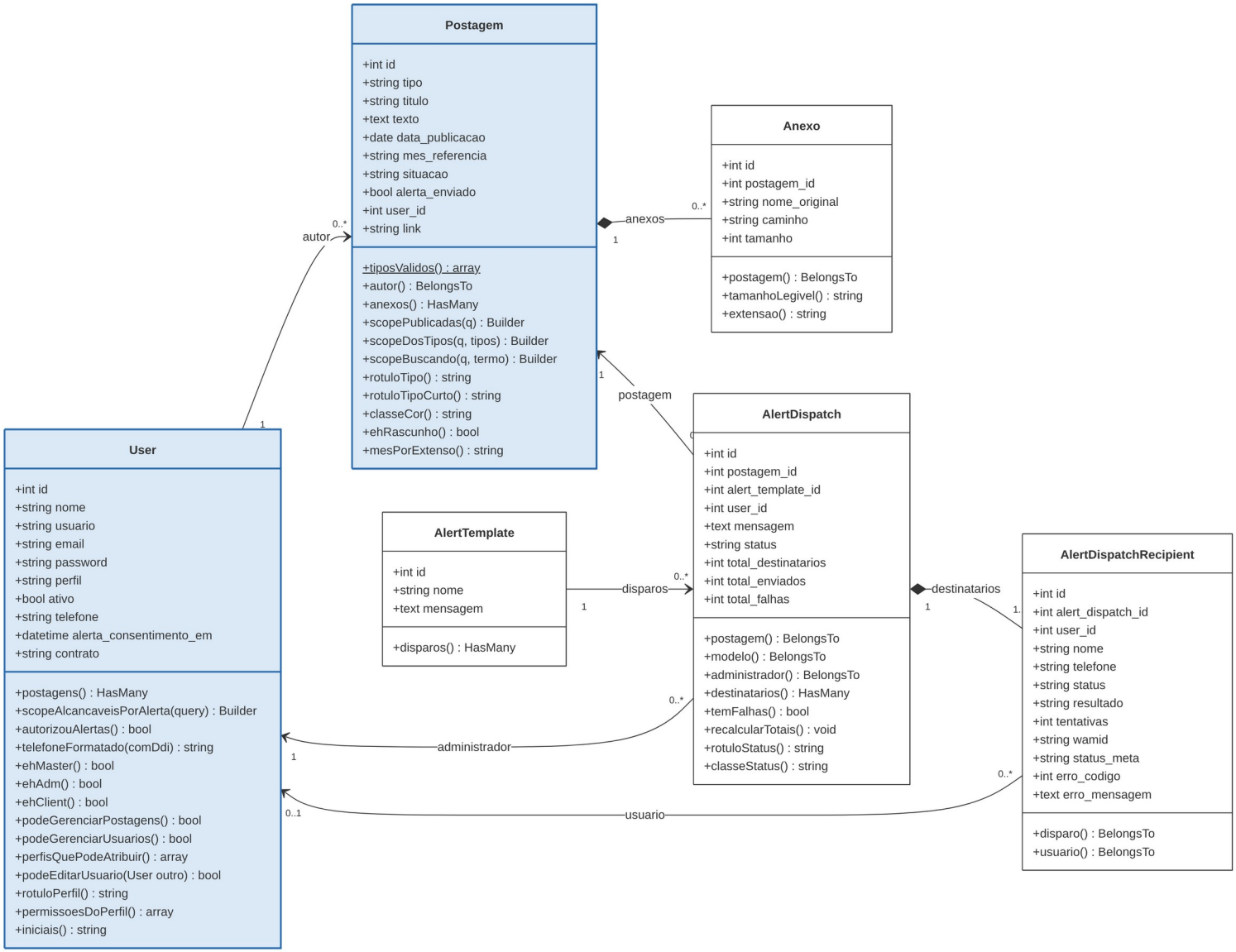
Diagrama de classes implementadas - Algoritmos e Dados

Diagramas gerados a partir do código-fonte do sistema (models, services, controllers e migrations). Estão representadas somente as classes efetivamente implementadas. Classes em azul foram criadas ou alteradas na Sprint 3 (módulo de relatórios).

1. Notação

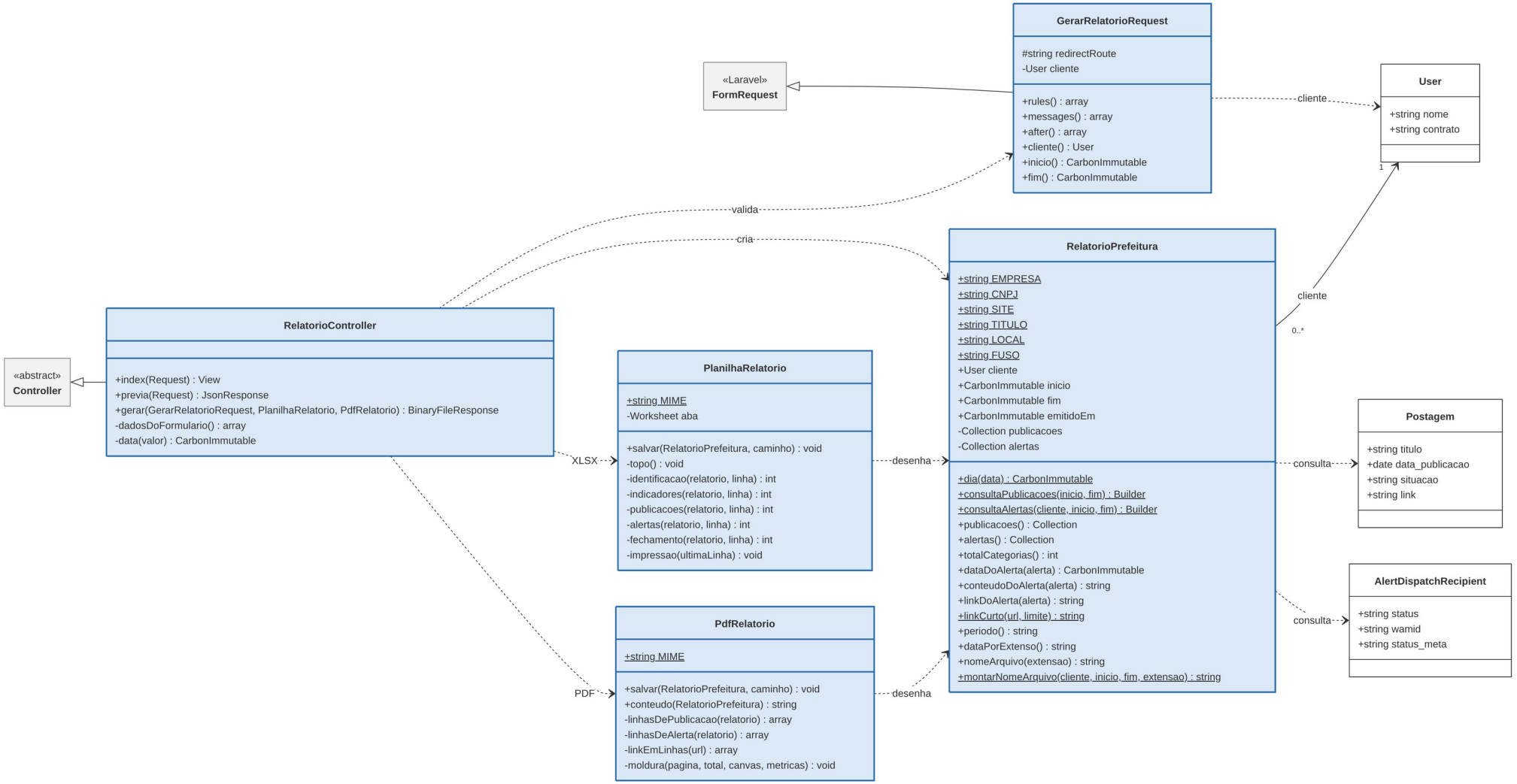
- Seta contínua com ponta aberta: associação navegável, com a cardinalidade em cada ponta.
- Losango preenchido: composição — a parte não existe sem o todo (exclusão em cascata no banco).
- Seta tracejada: dependência — a classe usa a outra, sem guardá-la.
- Triângulo vazio contínuo: herança. Triângulo vazio tracejado: implementação de interface.
- Visibilidade: + público, - privado, # protegido. Membro sublinhado: estático.
- Todas as entidades herdam de Model (Laravel); User herda de Authenticatable. Essas heranças estão omitidas no diagrama de domínio para deixá-lo legível. Todas as entidades têm também created_at e updated_at.

2. Diagrama de classes — modelo de domínio (entidades)



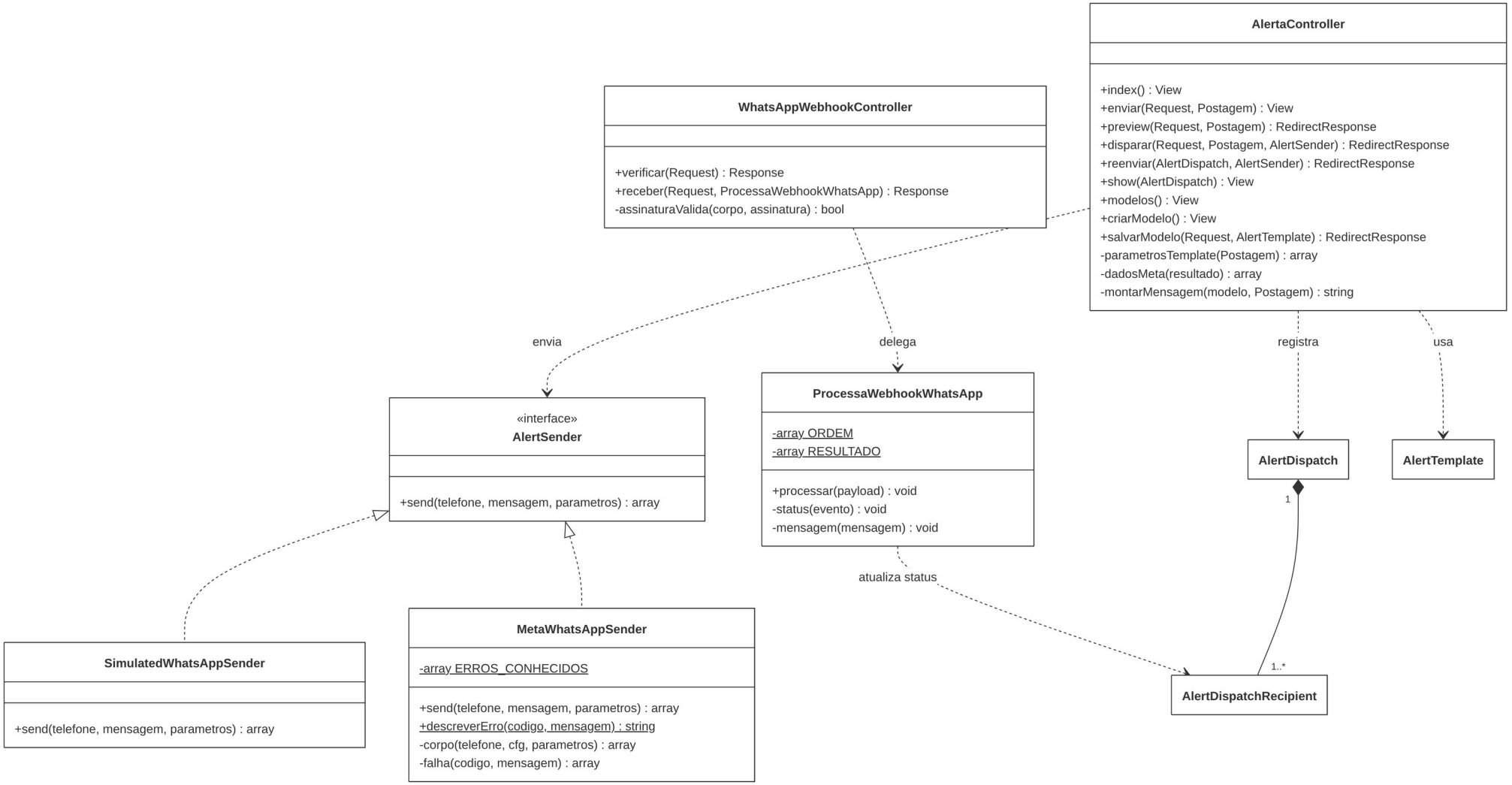
Entidades persistidas no banco. User.contrato e Postagem.link foram adicionados na Sprint 3.

3. Diagrama de classes — módulo de relatórios (Sprint 3)



User, Postagem e AlertDispatchRecipient aparecem resumidos, só com os atributos usados pelo relatório.

4. Diagrama de classes — envio de alertas por WhatsApp (Sprint 2)



O envio é escolhido pela configuração WHATSAPP_DRIVER: simulado ou Meta (WhatsApp Cloud API).

5. Classes

Número	Classe	Tipo	Responsabilidade	Sprint 3
1	User	Entidade	Usuário do sistema nos perfis Master, Administrador e Cliente (prefeitura). Guarda permissões, telefone e consentimento para alertas e, para o Cliente, o contrato.	Alterada (contrato)
2	Postagem	Entidade	Publicação do portal (edital, programa, informe etc.), com tipo, situação e link da fonte.	Alterada (link)
3	Anexo	Entidade	Arquivo anexado a uma publicação. A tabela existe, mas o upload ainda não foi implementado.	—
4	AlertTemplate	Entidade	Modelo de mensagem do alerta.	—
5	AlertDispatch	Entidade	Um disparo de alerta sobre uma publicação, com status e totais.	Usada (data do envio)
6	AlertDispatchRecipient	Entidade (classe associativa)	Destinatário de um disparo e resultado da entrega, incluindo o rastreio da Meta.	Usada (define o alerta entregue)
7	RelatorioPrefeitura	Serviço	Monta os dados do relatório: publicações do período e alertas entregues ao cliente, textos do cabeçalho e nome do arquivo.	Nova
8	PlanilhaRelatorio	Serviço	Gera o relatório em XLSX a partir de um RelatorioPrefeitura.	Nova
9	PdfRelatorio	Serviço	Gera o relatório em PDF a partir de um RelatorioPrefeitura.	Nova
10	RelatorioController	Controlador	Abre a tela de geração, devolve a prévia e entrega o arquivo para download.	Nova
11	GerarRelatorioRequest	Validação	Valida cliente (perfil Cliente com contrato), período e formato.	Nova
12	AlertSender	Interface	Contrato de envio de alerta: send(telefone, mensagem, parametros).	—
13	SimulatedWhatsAppSender	Serviço	Envio simulado, sem sair do servidor.	—
14	MetaWhatsAppSender	Serviço	Envio real pela WhatsApp Cloud API, por template aprovado.	—
15	ProcessaWebhookWhatsApp	Serviço	Processa os eventos da Meta e atualiza o status de cada destinatário.	Indireta (gera status_meta)
16	AlertaController	Controlador	Envio, reenvio, histórico e modelos de alerta.	—
17	WhatsAppWebhookController	Controlador	Verificação e recebimento do webhook da Meta, com conferência de assinatura.	—

Classes de apoio não representadas nos diagramas: AuthController, PainelController, PostagemController, UsuarioController e ContaController (telas), e Telefone, SanitizadorHtml e Asset (utilitários).

6. Relacionamentos

Número	Origem	Destino	Tipo	Cardinalidade	Descrição
1	User	Postagem	Associação	1 : 0..*	Um usuário é autor de várias publicações (postagens.user_id).
2	Postagem	Anexo	Composição	1 : 0..*	O anexo pertence à publicação e é excluído com ela.
3	AlertDispatch	Postagem	Associação	0..* : 1	Cada disparo é sobre uma publicação. Navegável só a partir do disparo.
4	AlertTemplate	AlertDispatch	Associação	1 : 0..*	Um modelo de mensagem é usado em vários disparos.
5	AlertDispatch	User	Associação	0..* : 1	Administrador que fez o disparo (alert_dispatches.user_id).

Número	Origem	Destino	Tipo	Cardinalidade	Descrição
6	AlertDispatch	AlertDispatchRecipient	Composição	1 : 1..*	Os destinatários existem só dentro do disparo.
7	AlertDispatchRecipient	User	Associação	0..* : 0..1	Usuário que recebeu. Se ele for excluído, o registro fica sem usuário e o histórico é mantido.
8	User	AlertDispatch	N : N (via classe associativa)	N : N	Um disparo vai para vários usuários e um usuário recebe vários disparos, por meio de AlertDispatchRecipient.
9	RelatorioPrefeitura	User	Associação	0..* : 1	Cada relatório é de um cliente (município e contrato).
10	RelatorioPrefeitura	Postagem, AlertDispatchRecipient	Dependência	—	Consulta as publicações do período e os alertas entregues ao cliente.
11	PlanilhaRelatorio, PdfRelatorio	RelatorioPrefeitura	Dependência	—	Recebem os dados prontos e os desenham no formato.
12	RelatorioController	GerarRelatorioRequest, RelatorioPrefeitura, PlanilhaRelatorio, PdfRelatorio	Dependência	—	Valida o pedido, monta o relatório e escolhe o formato.
13	SimulatedWhatsAppSender, MetaWhatsAppSender	AlertSender	Realização	—	Implementam a interface de envio.
14	AlertaController	AlertSender, AlertDispatch, AlertTemplate	Dependência	—	Envia pelo AlertSender e registra o disparo.
15	WhatsAppWebhookController	ProcessaWebhookWhatsApp	Dependência	—	Delega o processamento dos eventos da Meta.
16	ProcessaWebhookWhatsApp	AlertDispatchRecipient	Dependência	—	Atualiza o status de entrega pelo wamid.
17	RelatorioController, AlertaController, …	Controller	Herança	—	Todos os controladores herdam de Controller.
18	GerarRelatorioRequest	FormRequest	Herança	—	Validação padrão do Laravel.

Observação: as relações 3 e 7 existem no banco (chave estrangeira), mas no código estão declaradas apenas no lado de AlertDispatch e AlertDispatchRecipient. Não há relação 1:1 no modelo atual.